

Data for Perfect Bloom first Sight

k (iocs/keys)= 20,000,000

scalable_bloom b1 >= 9.2 bits per key

scalable_bloom b2 >= 19.2 bit per key

k1 (b1 hash functions)= 7

k2 (hash into vec slot) = 1

k3 (b2 hash functions) = 13

const vec_length= 4096

• The outer bloom with 20 million keys and a bit ratio of 9.2 bits per key and 7 hash functions has a false positive rate of most 0.91% p1.

• Hashing into the vector with 20 mill keys for 4096 slots is 4882 keys per slot, aka approximately 4882 keys disrtibuted to the sub bloom filters b2.

• Each slot in the vector will store a bloom filter. It's bit count will be relatively quite large 19.2 per key, this concept allows us to scale granularly to ensure false positives are controlled. The idea is inspired by perfect hashing.

• 4883 keys in a buckey with 93,7536 bits and 13 hash function results in 0.012% false positive rate p2.

• Combined p1 *p2 gives us 0.00011% fale positive rate, 1 in 900,000.

Space complexity analysis: Stores one big bloom, 4096 smaller ones, one vec, variable constants, ioc traffic.

OUTER BLOOM (b1)

20,000,000 keys * 9.2 bit/key = 184,000,000 bits

184,000,000 %8 = 23, 000,000 bytes

23 % 1,048,576 = 21.93 mb

INNER BLOOMS (B2)

4,883 KEYS * 19.2 = 93,753.6 bits per bucketbits per bucket

93,573.6 % 8 = 11,719.2 bytes per bucket

11,719.2 * 4096 = 48,002,611.2 bytes

48,002,611.2 % 1,048,576 = 45.8 mb

VECTOR

4096 * 8 = 32,768 = 0.03 mb

Total memory: 67.76 MB

Time compexity analysis: All main operations would be hash computations.

OUTER BLOOM (b1)

k1 = 7 hash functions

O(7) constant time

HASH INTO VECTOR

k2 = 1

O(1) constant time

INNER BLOOMS (b2)

k3 = 13

O(13) constant time

WORST CASE: OUTER + VEC + INNER = O(21) = O(1)

```
graph LR
    A([IOC/key trigegrs handler]) --> B[pass key through k1 functions into b1 bloom]
    B --> C{check b1 results}
    C -- No --> D[Mark for insertion not lookup]
    C -- Yes --> E[Hash key into vec with k2]
    E --> F[Pass key though k3 functions into b2 bloom]
    F --> G{Check b2 results}
    G -- No --> H([Triggers bloom Insert function])
    G -- Yes --> I([return key exists])
    D --> E
    H -- "if marked for insertion" --> E
```

The flowchart illustrates the lookup process for a key in the Perfect Bloom structure. It starts with an 'IOC/key trigegrs handler' (green oval) leading to 'pass key through k1 functions into b1 bloom' (blue rectangle). A decision diamond 'check b1 results' follows. If the result is 'No', it leads to 'Mark for insertion not lookup' (blue rectangle), which then leads to 'Hash key into vec with k2' (blue rectangle). If the result is 'Yes', it leads directly to 'Hash key into vec with k2'. From 'Hash key into vec with k2', the process moves to 'Pass key though k3 functions into b2 bloom' (blue rectangle). Another decision diamond 'Check b2 results' follows. If 'No', it triggers 'Triggers bloom Insert function' (green oval). If 'Yes', it leads to 'return key exists' (green oval). There is also a feedback loop from 'Triggers bloom Insert function' back to 'Hash key into vec with k2' labeled 'if marked for insertion'.

Perfect bloom for first sight

The diagram illustrates the Perfect Bloom structure for the first sight. It shows a hierarchy of Bloom filters. At the top is the 'Outer Bloom Filter (B1)' with '9.2 bits/n, 7 hash function'. An arrow points down to the 'Hash Into Vector' stage, which consists of '4096 slots/B2 blooms, 1 hash function'. This stage is shown as a large cyan rectangle with vertical lines representing slots. Below this, two arrows point to 'Inner Bloom Filter (B2)' structures. The left one is labeled 'Slot 0' and the right one is labeled 'Slot i'. Each 'Inner Bloom Filter (B2)' has '19.2 bits/n/4096, 13 hash function'.